

A Faster Attack on LFSR Recurrences

October 27, 2025

Timothy Tarter
Arsenii Herasymov
Garrett Priebracha

James Madison University
Madison Institute for Mathematical Finance
<https://github.com/Tartertx/LFSR-Research>

Abstract

Linear Feedback Shift Registers (LFSR) are a type of cryptographic algorithm used to generate pseudo-random numbers, encode binary plaintext, and make digital circuits efficient. Cryptographers often seek to find the minimal recurrence of an LFSR sequence output in order to reverse engineer the digital circuit behind the encryption process of a ciphertext. Heretofore, the best algorithm for accomplishing this task was the Berlekamp-Massey algorithm (and its derivatives), which can take a partial segment of a sequence and construct a recurrence polynomial in $O(n^2)$ time. In this paper, we present the Tarter-Herasymov-Priebracha algorithm which requires the full ciphertext sequence but accomplishes the task in $O(n + \log(n))$ time, a significant improvement over the status quo. That said, due to the assumption of perfect information, the THP algorithm is ideal for use cases where one has received a complete ciphertext sequence without interference, but the length of the ciphertext is infeasibly large to attack with Berlekamp-Massey.

1 Introduction

As stated in the abstract, Linear Feedback Shift Registers (or LFSR) are a cryptographic algorithm used to encode binary plaintext. When an LFSR system is given a string of plaintext (typically in binary), it uses a recurrence relation (a polynomial with binary coefficients) to create a recurring string of ciphertext. Previously, the primary method of finding the recurrence polynomial has been the Berlekamp-Massey algorithm, which builds systems of linear equations of length n , and checks the determinant of the coefficient matrix to see if it yields a feasible solution. Then, the system with the largest value of n which admits a determinant of 1 provides exactly the minimum recurrence polynomial.

The Berlekamp-Massey algorithm has many nice features. For example, if a ciphertext is incomplete for some reason, the Berlekamp-Massey algorithm has no problem finding the recurrence relation, provided the sequence repeats itself at least once. That said, the Berlekamp-Massey

algorithm has issues with strings of ciphertext which are incredibly long. Essentially, the algorithm has $O(n^2)$ time complexity with a leading constant of $2\lfloor \frac{n^2}{4} \rfloor$, rendering it to be less helpful as strings of ciphertext approach 10^6 many bits.

This issue is the precise problem we hope to solve: is there a way to get linear or near-linear time complexity? As it turns out, if we are given a complete string (i.e., an unchopped string with no noise from transmission), we can use a simple divisibility argument to find the recurrence in $O(n + \log(n))$ time. At that point, it is trivial to build the associated LFSR polynomial.

2 Tarter-Herasymov-Priebracha (THP) Algorithm

1. Identify n = length of string
2. Identify the proper factors of n , including multiplicities
3. Pick k = largest proper factor
4. Chop the last k digits off of the string, iterate $\frac{n}{k}$ times
5. Check that string against the full string (ciphertext)
6. If they match, store the last k digits as a potential option, if not, discard it
7. Repeat for all proper factors
8. The minimal recurrence is the minimum of the lengths of recurrences in the list of options

2.1 Proof of Algorithm

2.1.1 Setup

Let β be a ciphertext with an unknown, minimal recurrence γ , and let $n = |\beta|$.

Let $\theta = \{\text{last 'k' digits of } n \mid |k| \mid n, k \neq 1, n, \text{ and } \bigoplus_{i=1}^{\frac{n}{|k|}} \text{last } k \text{ digits of } \beta = \beta\}$.

2.1.2 Statement

We want to show that $\gamma \in \theta$ and we want to show that there does not exist something smaller than γ in θ .

2.1.3 Formalism

Since $|\gamma|$ divides n by definition, our algorithm will identify it as a potential k . Since it is also a recurrence, it can be reconstructed into β by concatenating it with itself $\frac{n}{|\gamma|}$ times, and thus is contained in θ .

To show that there is not a smaller element in θ than γ , we assume there exists $\eta \in \theta$ with $|\eta| < |\gamma|$. But if $\eta \in \theta$, then η can be reconstructed as a recurrence of β by concatenating itself $\frac{n}{|\eta|}$ times, and is therefore a smaller recurrence than γ , which we assumed to be the minimal recurrence. Therefore, there does not exist such an η , and $\min \theta = \gamma$.

□

3 Complexity

We claim that the complexity of this algorithm is $O(n + \log(n))$.

3.1 Proof of Complexity

In our code, we have one ‘for’ loop which iterates n -many times to find factors (not necessarily prime factors, otherwise we would use the Sieve of Erasthones). Then we have a second ‘for loop’ for which we iterate through those factors, of which there are roughly $\log(n)$ many. Therefore, we perform $n + \log(n)$ many computations to implement our algorithm since these loops aren’t nested.

4 Implementation & Applications

4.1 Code

Our code implementation is as follows (also available on GitHub):

```

31 def decode(sequence, tests, l):
32     n = len(sequence) #get length
33     factors = []
34     for j in range(n-2): #find factors ### O(n)
35         if n % (j+2) == 0:
36             factors += [n/(j+2)]
37     factors.sort()
38     options = {}
39     for i in range(len(factors)-1,1,-1): #####O(log(n))
40         rec = sequence[int(n - factors[i]):n] #isolate the recurrence
41         trial = f'{rec}'*int(n//factors[i]) #concat to make the trial
42         if trial == str(tests.loc[l,'Test Case']): #tests if the trial works
43             options.update({len(rec):rec})
44     return options[min(options)]

```

Figure 1: Algorithm Implementation

4.2 Efficiency Results

The coefficient of our time complexity is shown to converge for $O(n + \log(n))$ as follows.

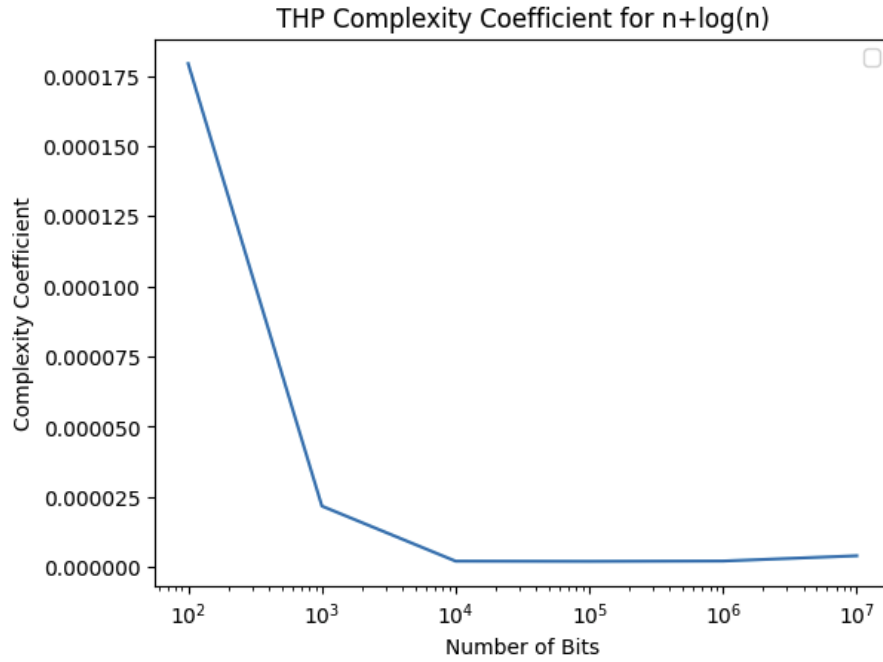


Figure 2: Convergence to Complexity Coefficient

The associated graph of computation time (in seconds) is as follows.

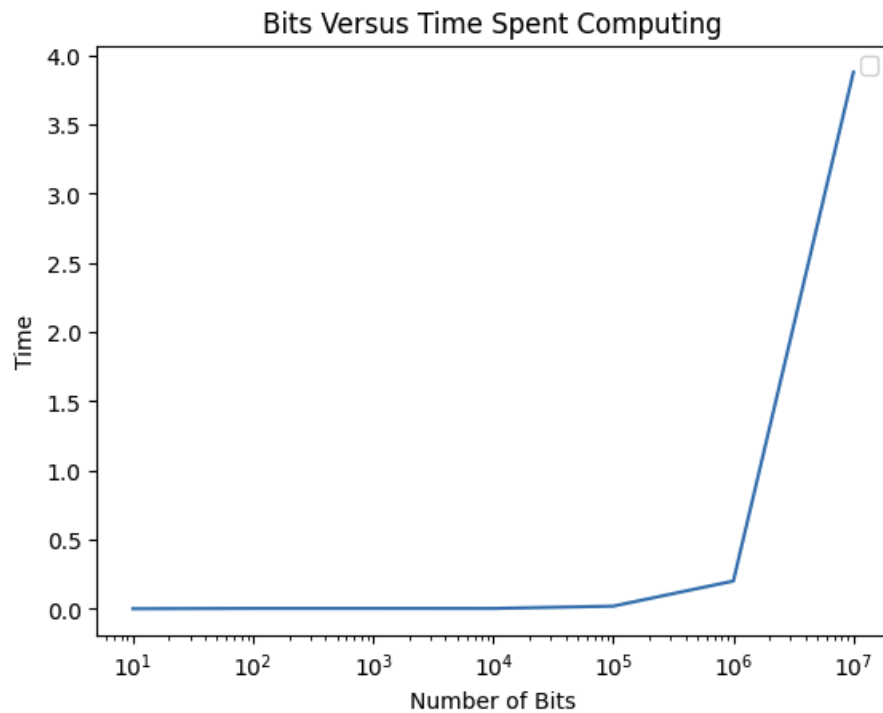


Figure 3: Our Results

The jump to 4 seconds seems extreme, but when we compare it to the Berlekamp-Massey algorithm for the same number of bits, it really doesn't seem bad at all!

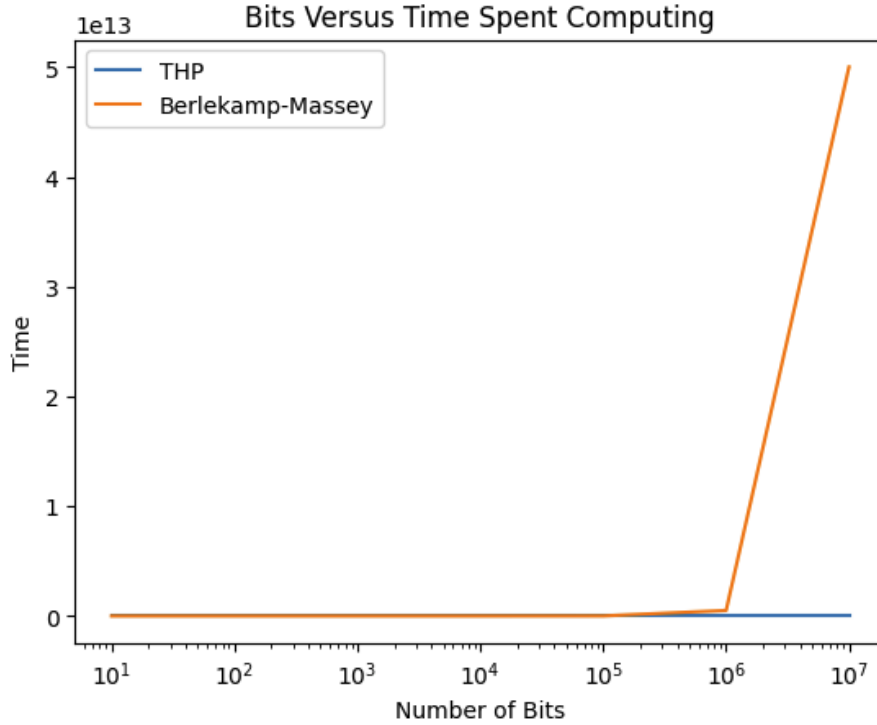


Figure 4: Our Results (Compared to Berlekamp Massey)

4.2.1 Differences From the Berlekamp-Massey Framework

While the Berlekamp-Massey algorithm can work with partial strings, an admitted hurdle for the use of the Tarter-Herasymov-Priebracha algorithm is that it requires full strings of ciphertext for a combinatorial attack to work. Additionally, the Berlekamp-Massey algorithm provides the recurrence polynomial which provides the recurrence - the THP algorithm provides the recurrence from which the recurrence polynomial can be derived. This mutual inverse property is ultimately irrelevant to the application of either algorithm to a problem.

The Berlekamp-Massey algorithm will work better when an LFSR sequence does not encounter additional noise in transmission between senders and receivers whereas the THP algorithm requires perfect information of the intended LFSR ciphertext.

Ultimately, both algorithms have their optimal use cases; the authors of this paper recommend the THP algorithm for instances where LFSR sequences are perfectly received but have computationally infeasible lengths.

Works Cited

Galbraith, S. D. (2007). Cryptography and coding: 11th IMA International Conference, Cirencester, UK, December 18-20, 2007: Proceedings. Springer.

Guibas, L. J., & Odlyzko, A. M. (1981). Periods in strings. *Journal of Combinatorial Theory, Series A*, 30(1), 19–42. [https://doi.org/10.1016/0097-3165\(81\)90038-8](https://doi.org/10.1016/0097-3165(81)90038-8)

Kreczmar, A., & Mirkowska, G. (1989). *Mathematical Foundations of Computer Science 1989: Porabka-Kozubnik, Poland August 28 - September 1, 1989 proceedings*. Springer-Verlag: Springer e-books.

Nikolaev, S. (2025). Advanced berlekamp–Massey method as a basis for finding periodicities in bitstreams. *Cybernetics and Systems Analysis*, 61(1), 3–10. <https://doi.org/10.1007/s10559-025-00739-1>

Tomlinson, M., Tjhai, C. J., Ambroze, M. A., Ahmed, M., & Jibril, M. (2017). *Error-correction coding and decoding bounds, codes, decoders, analysis and applications*. Springer International Publishing.

Wu, Y. (2023). High-speed LFSR decoder architectures for BCH and GII codes. *IEEE Journal on Selected Areas in Information Theory*, 4, 331–350. <https://doi.org/10.1109/jsait.2023.3304235>

Yu, K., & Hung, L. (1995). On binary recurrence sequences. *Indagationes Mathematicae*, 6(3), 341–354. [https://doi.org/10.1016/0019-3577\(95\)93201-k](https://doi.org/10.1016/0019-3577(95)93201-k)